DentComm Integration

1. Overview

1.1 Description

DentComm Integration connects **DentComm** (dental practice management system by DentTracks) with **Newo Platform**.

It enables:

- Checking real-time appointment availability with LLM-based appointment type matching
- Booking dental appointments with automatic new vs existing patient detection
- Slot caching for seamless booking after availability check
- Configurable booking window for forward-looking availability

This integration is designed for **dental clinic administrators and front-desk teams** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a patient asks about available slots → Resolve visit reason to appointment type via LLM, return available slots
- When a patient wants to book an appointment → Check if patient exists by phone, create appointment with cached slot data
- When a new patient books → Automatically detect and create with full patient details (name, DOB, email, phone)
- When a returning patient books → Find existing record by phone and use patient ID

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✅	With LLM-based appointment type resolution
Book Appointment	✅	Auto-detects new vs existing patient by phone
Appointment Type Matching	✅	LLM matches visit_reason to available types (Gemini)
Slot Caching	✅	Caches slots per session, cleared after booking
Patient Lookup	✅	By phone number
Configurable Booking Window	✅	Default 14 days, configurable via attribute
Timezone Handling	✅	Converts to business timezone for display
Cancel Appointment	❌	Not implemented in current version
Reschedule Appointment	❌	Not implemented
Multi-Location Support	❌	Single clinic ID per instance
Existing Client History	❌	No appointment history retrieval

2. Requirements

Before installation:

- Active **DentComm** account with API access
- **DentComm API Key** (`x-api-key` header)
- **Clinic ID** (identifies the dental practice)
- DentComm API base URL (default: `https://api.dev.denttracks.com/api/dentcomm`)

3. How to Setup

3.1 Step 1 — Obtain Credentials from DentComm

1. Log in to your **DentComm** / DentTracks account
2. Navigate to the API or developer settings section
3. Obtain the following:
    - **API Key** for authentication
    - **Clinic ID** for your practice
4. Store credentials securely

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`dentcomm_api_key`	✅	API key for DentComm authentication (`x-api-key` header)
`dentcomm_clinic_id`	✅	Clinic/practice identifier (`clinic-id` header)
`dentcomm_base_url`	❌	API base URL (default: `https://api.dev.denttracks.com/api/dentcomm`)
`dentcomm_booking_window`	❌	Days ahead to show availability (default: `14`)
`dentcomm_override_agent_attributes`	❌	Override superagent booking settings (default: `True`)

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Validates the API credentials
    - Creates the HTTP connector for DentComm
    - Injects booking and availability payload schemas for the agent

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow

- An **Action** is the API operation performed in DentComm

Available Triggers

Trigger	Event	Description
Check Availability	`get_available_slots`	When the agent needs to look up open slots
Book Appointment	`book_slot`	When the agent confirms a booking
Setup	`project_publish_finish`	Initializes integration on deploy

Available Actions

- **Get Appointment Types** — Fetches available types (GET `/pms-actions/appointment-type`)
- **Get Available Slots** — Queries open slots by type and date range (GET `/pms-actions/appointment/available-slot`)
- **Search Patient** — Finds patient by phone number (GET `/patient/by-number`)
- **Create Appointment** — Books an appointment slot (POST `/pms-actions/appointment`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as DentComm Integration
    participant API as DentComm API

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>I: Create connector, inject schemas
    I->>A: Booking tools configured

    Note over U,API: Availability Check
    U->>A: "Any slots for a cleaning next week?"
    A->>I: get_available_slots (date, visit_reason)
    I->>API: GET /pms-actions/appointment-type
    API-->>I: Appointment types list
    I->>I: LLM match visit_reason to type
    I->>API: GET /pms-actions/appointment/available-slot
    API-->>I: Available slots
    I->>I: Convert to timezone, cache slots
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:30 PM please"
    A->>I: book_slot (customerInfo, dateTime)
    I->>API: GET /patient/by-number?mobile=(phone)
    API-->>I: Patient result
    I->>I: Resolve slot from cache
    alt New patient
```

```
        I->>API: POST /pms-actions/appointment (isNewPatient: true, patient details)
    else Existing patient
        I->>API: POST /pms-actions/appointment (isNewPatient: false, patientId)
    end
    API-->>I: pmsId (appointment ID)
    I->>I: Clear slot cache
    I->>A: result_success (appointment_id)
    A->>U: "Your appointment is confirmed!"
```

AvailabilityFlow

```
flowchart TD
    E["get_available_slots"] --> EX["Extract payload:<br>date, time, visit_reason"]
    EX --> TYPES["GET /pms-actions/appointment-type"]
    TYPES --> OK1{"HTTP 200?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| PARSE["Parse types:<br>id, serviceName, description"]
    PARSE --> LLM["LLM match visit_reason<br>to appointment type<br>(Gemini,
temp=0.0)"]
    LLM --> RANGE["Calculate date range:<br>date → date + booking_window"]
    RANGE --> SLOTS["GET /available-slot<br>?appointmentTypeId={id}
<br>&startDate&endDate"]
    SLOTS --> OK2{"HTTP 200?"}
    OK2 -->|"No"| ERR
    OK2 -->|"Yes"| TZ["Convert slots to<br>business timezone"]
    TZ --> DEDUP["Deduplicate by<br>display time"]
    DEDUP --> CACHE["Cache slots in persona:<br>{date time} → {opId, provId,
<br>duration, originalStart, apptTypeId}"]
    CACHE --> OUT["result_success<br>{availability[]}"]
```

BookingFlow

```
flowchart TD
    E["book_slot"] --> EX["Extract customerInfo:<br>name, phone, email, DOB"]
    EX --> SEARCH["GET /patient/by-number<br>?mobile={phone}"]
    SEARCH --> OK1{"HTTP 200?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| FOUND{"Patient<br>found?"}
    FOUND -->|"Yes"| PID["Extract patient_id"]
    FOUND -->|"No"| NEW["isNewPatient = true"]
    PID --> CACHE["Resolve slot<br>from cache"]
    NEW --> CACHE
    CACHE --> HIT{"Cache<br>hit?"}
    HIT -->|"No"| ERR
    HIT -->|"Yes"| API["POST /pms-actions/appointment<br>{appointmentDate,
apptTypeId,<br>duration, opId, providerId,<br>isNewPatient, patient data}"]
    API --> OK2{"HTTP 200?"}
    OK2 -->|"No"| ERR
```

```
    OK2 -->|"Yes"| CLEAR["Clear slot cache"]
    CLEAR --> OUT["result_success<br>{appointment_id: pmsId}"]
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. The AvailabilityFlow also has a dedicated test skill triggered via the `integration_test` webhook event.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify schemas are injected (check for `booking_payload_schema_template` attribute)
2. **Availability**: Ask the agent "What slots are available for a cleaning?" — verify appointment types are resolved and slots returned
3. **Booking (new patient)**: Book with a new phone number — verify `isNewPatient: true` and patient details sent
4. **Booking (existing patient)**: Book with a known phone number — verify `patientId` used instead of full details

If no action occurs:

- Ensure `dentcomm_api_key` and `dentcomm_clinic_id` are set correctly
- Verify `dentcomm_base_url` points to the correct endpoint
- Confirm the integration was re-published after configuration changes
- Check that the slot cache contains data (availability must be checked before booking)

> *Note: This integration performs real operations in DentComm. Appointments created through the agent are reflected in the live dental practice management system.*

5. FAQ

Q: Does the integration support cancellations or rescheduling? A: No. The current version supports availability checking and booking only. To cancel or reschedule, use the DentComm dashboard directly.

Q: Can I connect multiple clinics? A: Each integration instance supports one clinic ID. For multiple clinics, create separate integration instances.

Q: How does appointment type matching work? A: When a patient describes their visit reason (e.g., "teeth cleaning", "checkup"), an LLM (Gemini) matches it to available appointment types from DentComm using zero-temperature generation for deterministic results. For ambiguous requests, the safest/most general option is selected.

Q: Why does the agent need to check availability before booking? A: The availability check caches slot metadata (operatory ID, provider ID, duration, original start time, appointment type ID) in a persona attribute. The booking flow retrieves this cached data to construct the appointment request. Without a prior availability check, the cache is empty and booking will fail.

Q: What patient data is required for a new patient booking? A: First name, last name, date of birth (YYYY-MM-DD), email, and phone number (E.164 format). For existing patients, only the phone number is needed for lookup.

Q: How is the booking window configured? A: The `dentcomm_booking_window` attribute controls how many days ahead slots are shown (default: 14 days). When a patient asks for availability on a specific date, slots are returned from that date through `date + booking_window` days.